

Programming Challenges

1. Numbers Class

Design a class `Numbers` that can be used to translate whole dollar amounts in the range 0 through 9999 into an English description of the number. For example, the number 713 would be translated into the string *seven hundred thirteen*, and 8203 would be translated into *eight thousand two hundred three*. The class should have a single integer member variable:

```
int number;
```

and a static array of `string` objects that specify how to translate key dollar amounts into the desired format. For example, you might use static strings such as

```
string lessThan20[20] = {"zero", "one", ..., "eighteen", "nineteen"};
string hundred = "hundred";
string thousand = "thousand";
```

The class should have a constructor that accepts a nonnegative integer and uses it to initialize the `Numbers` object. It should have a member function `print()` that prints the English description of the `Numbers` object. Demonstrate the class by writing a main program that asks the user to enter a number in the proper range and then prints out its English description.

2. Day of the Year

Assuming that a year has 365 days, write a class named `DayOfYear` that takes an integer representing a day of the year and translates it to a string consisting of the month followed by day of the month. For example,

Day 2 would be *January 2*.

Day 32 would be *February 1*.

Day 365 would be *December 31*.

The constructor for the class should take as parameter an integer representing the day of the year, and the class should have a member function `print()` that prints the day in the month–day format. The class should have an integer member variable to represent the day and should have static member variables holding `string` objects that can be used to assist in the translation from the integer format to the month–day format.

Test your class by inputting various integers representing days and printing out their representation in the month–day format.

3. Day of the Year Modification

Modify the `DayOfYear` class, written in Programming Challenge 2, to add a constructor that takes two parameters: a `string` object representing a month and an integer in the range 0 through 31 representing the day of the month. The constructor should then initialize the integer member of the class to represent the day specified by the month and day of month parameters. The constructor should terminate the program with an appropriate error message if the number entered for a day is outside the range of days for the month given.

Add the following overloaded operators:

- ++ prefix and postfix increment operators.** These operators should modify the `DayOfYear` object so that it represents the next day. If the day is already the end of the year, the new value of the object will represent the first day of the year.
- prefix and postfix decrement operators.** These operators should modify the `DayOfYear` object so that it represents the previous day. If the day is already the first day of the year, the new value of the object will represent the last day of the year.

4. NumDays Class



VideoNote
Solving the
NumDays
Problem

Design a class called `NumDays`. The class's purpose is to store a value that represents a number of work hours and convert it to a number of days. For example, 8 hours would be converted to 1 day, 12 hours would be converted to 1.5 days, and 18 hours would be converted to 2.25 days. The class should have a constructor that accepts a number of hours, as well as member functions for storing and retrieving the hours and days. The class should also have the following overloaded operators:

- + Addition operator.** When two `NumDays` objects are added together, the overloaded `+` operator should return the sum of the two objects' hours members.
- Subtraction operator.** When one `NumDays` object is subtracted from another, the overloaded `-` operator should return the difference of the two objects' hours members.
- ++ Prefix and postfix increment operators.** These operators should increment the number of hours stored in the object. When incremented, the number of days should be automatically recalculated.
- Prefix and postfix decrement operators.** These operators should decrement the number of hours stored in the object. When decremented, the number of days should be automatically recalculated.

5. Time Off



NOTE: This assignment assumes you have already completed Programming Challenge 4.

Design a class named `TimeOff`. The purpose of the class is to track an employee's sick leave, vacation, and unpaid time off. It should have, as members, the following instances of the `NumDays` class described in Programming Challenge 4:

<code>maxSickDays</code>	A <code>NumDays</code> object that records the maximum number of days of sick leave the employee may take.
<code>sickTaken</code>	A <code>NumDays</code> object that records the number of days of sick leave the employee has already taken.
<code>maxVacation</code>	A <code>NumDays</code> object that records the maximum number of days of paid vacation the employee may take.
<code>vacTaken</code>	A <code>NumDays</code> object that records the number of days of paid vacation the employee has already taken.

<code>maxUnpaid</code>	A <code>NumDays</code> object that records the maximum number of days of unpaid vacation the employee may take.
<code>unpaidTaken</code>	A <code>NumDays</code> object that records the number of days of unpaid leave the employee has taken.

Additionally, the class should have members for holding the employee's name and identification number. It should have an appropriate constructor and member functions for storing and retrieving data in any of the member objects.

Input Validation: Company policy states that an employee may not accumulate more than 240 hours of paid vacation. The class should not allow the `maxVacation` object to store a value greater than this amount.

6. Personnel Report



NOTE: This assignment assumes you have already completed Programming Challenges 4 and 5.

Write a program that uses an instance of the `TimeOff` class you designed in Programming Challenge 5. The program should ask the user to enter the number of months an employee has worked for the company. It should then use the `TimeOff` object to calculate and display the employee's maximum number of sick leave and vacation days. Employees earn 12 hours of vacation leave and 8 hours of sick leave per month.

7. Month Class

Design a class named `Month`. The class should have the following private members:

- `name` A `string` object that holds the name of a month, such as "January," "February," etc.
- `monthNumber` An integer variable that holds the number of the month. For example, January would be 1, February would be 2, etc. Valid values for this variable are 1 through 12.

In addition, provide the following member functions:

- A default constructor that sets `monthNumber` to 1 and `name` to "January."
- A constructor that accepts the name of the month as an argument. It should set `name` to the value passed as the argument and set `monthNumber` to the correct value.
- A constructor that accepts the number of the month as an argument. It should set `monthNumber` to the value passed as the argument and set `name` to the correct month name.
- Appropriate set and get functions for the `name` and `monthNumber` member variables.
- Prefix and postfix overloaded `++` operator functions that increment `monthNumber` and set `name` to the name of next month. If `monthNumber` is set to 12 when these functions execute, they should set `monthNumber` to 1 and `name` to "January."
- Prefix and postfix overloaded `--` operator functions that decrement `monthNumber` and set `name` to the name of previous month. If `monthNumber` is set to 1 when these functions execute, they should set `monthNumber` to 12 and `name` to "December."

Also, you should overload `cout`'s `<<` operator and `cin`'s `>>` operator to work with the `Month` class. Demonstrate the class in a program.

8. Date Class Modification

Modify the `Date` class in Programming Challenge 1 of Chapter 13. The new version should have the following overloaded operators:

- `++` *Prefix and postfix increment operators.* These operators should increment the object's day member.
- `--` *Prefix and postfix decrement operators.* These operators should decrement the object's day member.
- `-` *Subtraction operator.* If one `Date` object is subtracted from another, the operator should give the number of days between the two dates. For example, if April 10, 2014 is subtracted from April 18, 2014, the result will be 8.
- `<<` *`cout`'s stream insertion operator.* This operator should cause the date to be displayed in the form
 April 18, 2014
- `>>` *`cin`'s stream extraction operator.* This operator should prompt the user for a date to be stored in a `Date` object.

The class should detect the following conditions and handle them accordingly:

- When a date is set to the last day of the month and incremented, it should become the first day of the following month.
- When a date is set to December 31 and incremented, it should become January 1 of the following year.
- When a day is set to the first day of the month and decremented, it should become the last day of the previous month.
- When a date is set to January 1 and decremented, it should become December 31 of the previous year.

Demonstrate the class's capabilities in a simple program.

Input Validation: The overloaded `>>` operator should not accept invalid dates. For example, the date 13/45/2014 should not be accepted.

9. FeetInches Modification

Modify the `FeetInches` class discussed in this chapter so it overloads the following operators:

`<=`
`>=`
`!=`

Demonstrate the class's capabilities in a simple program.

10. Corporate Sales

A corporation has six divisions, each responsible for sales to different geographic locations. Design a `DivSales` class that keeps sales data for a division, with the following members:

- An array with four elements for holding four quarters of sales figures for the division.
- A private static variable for holding the total corporate sales for all divisions for the entire year.

- A member function that takes four arguments, each assumed to be the sales for a quarter. The value of the arguments should be copied into the array that holds the sales data. The total of the four arguments should be added to the static variable that holds the total yearly corporate sales.
- A function that takes an integer argument within the range of 0–3. The argument is to be used as a subscript into the division quarterly sales array. The function should return the value of the array element with that subscript.

Write a program that creates an array of six `DivSales` objects. The program should ask the user to enter the sales for four quarters for each division. After the data are entered, the program should display a table showing the division sales for each quarter. The program should then display the total corporate sales for the year.

Input Validation: Only accept positive values for quarterly sales figures.

11. `FeetInches` Class Copy Constructor and `multiply` Function

Add a copy constructor to the `FeetInches` class. This constructor should accept a `FeetInches` object as an argument. The constructor should assign to the `feet` attribute the value in the argument's `feet` attribute, and assign to the `inches` attribute the value in the argument's `inches` attribute. As a result, the new object will be a copy of the argument object.

Next, add a `multiply` member function to the `FeetInches` class. The `multiply` function should accept a `FeetInches` object as an argument. The argument object's `feet` and `inches` attributes will be multiplied by the calling object's `feet` and `inches` attributes, and a `FeetInches` object containing the result will be returned.

12. `LandTract` Class

Make a `LandTract` class that is composed of two `FeetInches` objects, one for the tract's length and one for the width. The class should have a member function that returns the tract's area. Demonstrate the class in a program that asks the user to enter the dimensions for two tracts of land. The program should display the area of each tract of land and indicate whether the tracts are of equal size.

13. Carpet Calculator

The Westfield Carpet Company has asked you to write an application that calculates the price of carpeting for rectangular rooms. To calculate the price, you multiply the area of the floor (width times length) by the price per square foot of carpet. For example, the area of floor that is 12 feet long and 10 feet wide is 120 square feet. To cover that floor with carpet that costs \$8 per square foot would cost \$960. ($12 \times 10 \times 8 = 960$.)

First, you should create a class named `RoomDimension` that has two `FeetInches` objects as attributes: one for the length of the room and one for the width. (You should use the version of the `FeetInches` class that you created in Programming Challenge 11 with the addition of a `multiply` member function. You can use this function to calculate the area of the room.) The `RoomDimension` class should have a member function that returns the area of the room as a `FeetInches` object.

Next, you should create a `RoomCarpet` class that has a `RoomDimension` object as an attribute. It should also have an attribute for the cost of the carpet per square foot. The `RoomCarpet` class should have a member function that returns the total cost of the carpet.

Once you have written these classes, use them in an application that asks the user to enter the dimensions of a room and the price per square foot of the desired carpeting. The application should display the total cost of the carpet.

14. Parking Ticket Simulator

For this assignment you will design a set of classes that work together to simulate a police officer issuing a parking ticket. The classes you should design are:

- **The `ParkedCar` Class:** This class should simulate a parked car. The class's responsibilities are:
 - To know the car's make, model, color, license number, and the number of minutes that the car has been parked
- **The `ParkingMeter` Class:** This class should simulate a parking meter. The class's only responsibility is:
 - To know the number of minutes of parking time that has been purchased
- **The `ParkingTicket` Class:** This class should simulate a parking ticket. The class's responsibilities are:
 - To report the make, model, color, and license number of the illegally parked car
 - To report the amount of the fine, which is \$25 for the first hour or part of an hour that the car is illegally parked, plus \$10 for every additional hour or part of an hour that the car is illegally parked
 - To report the name and badge number of the police officer issuing the ticket
- **The `PoliceOfficer` Class:** This class should simulate a police officer inspecting parked cars. The class's responsibilities are:
 - To know the police officer's name and badge number
 - To examine a `ParkedCar` object and a `ParkingMeter` object, and determine whether the car's time has expired
 - To issue a parking ticket (generate a `ParkingTicket` object) if the car's time has expired

Write a program that demonstrates how these classes collaborate.

15. Car Instrument Simulator

For this assignment you will design a set of classes that work together to simulate a car's fuel gauge and odometer. The classes you will design are:

- **The `FuelGauge` Class:** This class will simulate a fuel gauge. Its responsibilities are:
 - To know the car's current amount of fuel, in gallons.
 - To report the car's current amount of fuel, in gallons.
 - To be able to increment the amount of fuel by 1 gallon. This simulates putting fuel in the car. (The car can hold a maximum of 15 gallons.)
 - To be able to decrement the amount of fuel by 1 gallon, if the amount of fuel is greater than 0 gallons. This simulates burning fuel as the car runs.
- **The `Odometer` Class:** This class will simulate the car's odometer. Its responsibilities are:
 - To know the car's current mileage.
 - To report the car's current mileage.

- To be able to increment the current mileage by 1 mile. The maximum mileage the odometer can store is 999,999 miles. When this amount is exceeded, the odometer resets the current mileage to 0.
- To be able to work with a `FuelGauge` object. It should decrease the `FuelGauge` object's current amount of fuel by 1 gallon for every 24 miles traveled. (The car's fuel economy is 24 miles per gallon.)

Demonstrate the classes by creating instances of each. Simulate filling the car up with fuel, and then run a loop that increments the odometer until the car runs out of fuel. During each loop iteration, print the car's current mileage and amount of fuel.